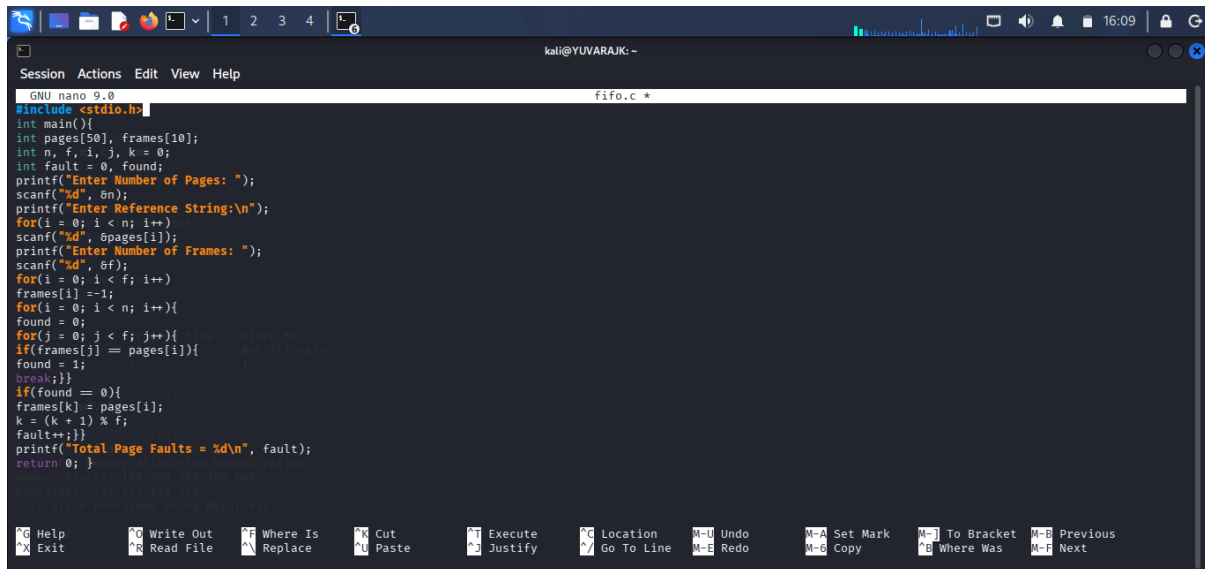


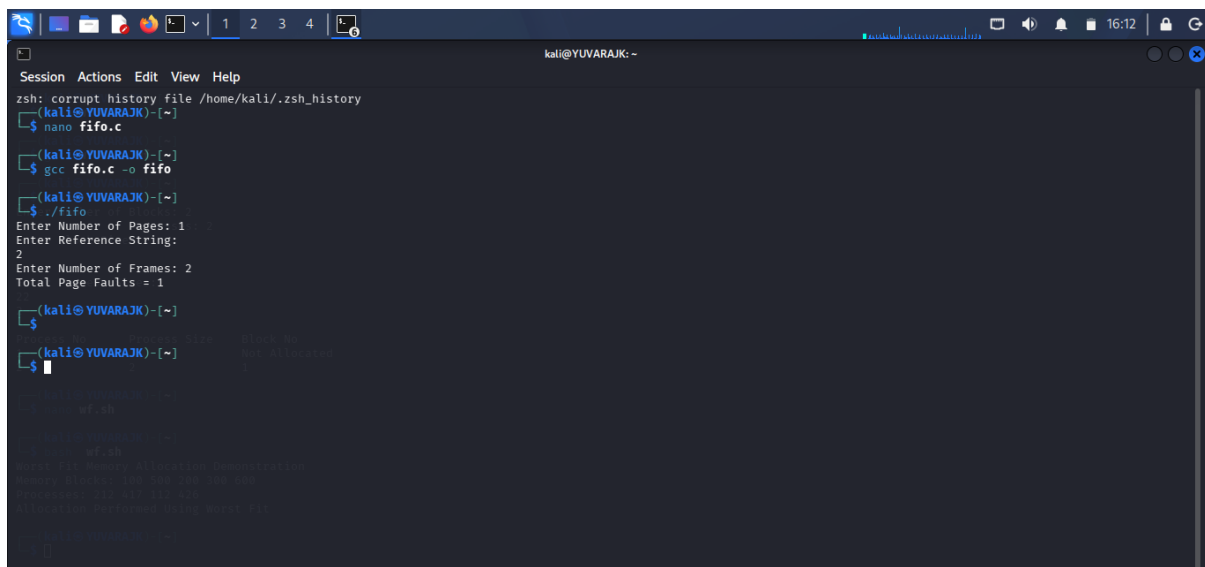
C PROGRAMMING : FIFO



```
GNU nano 9.0 fifo.c *
#include <stdio.h>

int main(){
    int pages[50], frames[10];
    int n, f, i, j, k = 0;
    int fault = 0, found;
    printf("Enter Number of Pages: ");
    scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");
    scanf("%d", &f);
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++){
        found = 0;
        for(j = 0; j < f; j++){
            if(frames[j] == pages[i]){
                found = 1;
                break;
            }
        }
        if(found == 0){
            frames[k] = pages[i];
            k = (k + 1) % f;
            fault++;
        }
    }
    printf("Total Page Faults = %d\n", fault);
    return 0; }
```

OUTPUT :



```
kali@YUVARAJK: ~
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano fifo.c
(kali@YUVARAJK)-[~]
$ gcc fifo.c -o fifo
(kali@YUVARAJK)-[~]
$ ./fifo
Enter Number of Pages: 1
Enter Reference String:
2
Enter Number of Frames: 2
Total Page Faults = 1
(kali@YUVARAJK)-[~]
$
```

SHELL PROGRAMMING : FIFO

```

GNU nano 9.0                                fifo.sh
#!/bin/bash
echo "FIFO Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
frames=3
echo "Reference String: ${pages[@]}"
echo "Frames: $frames"
echo "FIFO Algorithm Executed"

# Memory Allocation Demonstration
# Initial state: all frames are free
# Pages to be allocated: 7 0 1 2 0 3 0 4 2 3 0 3 2
# Total pages: 13
# Total frames: 3

# Memory Allocation Process
# 1. Page 7: Free frames (3) -> 1 frame occupied
# 2. Page 0: Free frames (2) -> 2 frames occupied
# 3. Page 1: Free frames (1) -> 3 frames occupied
# 4. Page 2: Free frames (0) -> 3 frames occupied
# 5. Page 0: Free frames (0) -> 3 frames occupied
# 6. Page 3: Free frames (0) -> 3 frames occupied
# 7. Page 0: Free frames (0) -> 3 frames occupied
# 8. Page 4: Free frames (0) -> 3 frames occupied
# 9. Page 2: Free frames (0) -> 3 frames occupied
# 10. Page 3: Free frames (0) -> 3 frames occupied
# 11. Page 0: Free frames (0) -> 3 frames occupied
# 12. Page 3: Free frames (0) -> 3 frames occupied
# 13. Page 2: Free frames (0) -> 3 frames occupied

# Memory Allocation Result
# Final state: all frames are occupied
# Pages in memory: 7 0 1
# Pages not in memory: 2 0 3 0 4 2 3 0 3 2
# Total pages not in memory: 10
# Total pages in memory: 3
# Total pages: 13
# Total frames: 3

```

OUTPUT :

The screenshot shows a Kali Linux terminal window titled "kali@YUVARAJK: ~". The user has executed the command `zsh: corrupt history file /home/kali/.zsh_history`. They then created a C program named `fifo.c` using `nano`, which implements a FIFO page replacement algorithm. The program takes three inputs: Number of Pages (1), Reference String (7 0 1 2 0 3 0 4 2 3 0 3 2), and Number of Frames (2). It outputs the number of page faults as 1. Subsequently, they created a shell script named `fifo.sh` using `nano`, which runs the C program and displays its output. The final output shown is:

```
FIFO Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Frames: 3
FIFO Algorithm Executed on Demonstration
Page Faults = 1
Total Page Faults = 1
```

C PROGRAMMING :LRU

```
GNU nano 9.0 lru.c *
#include <stdio.h>
int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j;
    int fault = 0, count = 0;
    int found, pos, min;
    printf("Enter Number of Pages: ");scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");scanf("%d", &f);
    for(i = 0; i < f; i++)frames[i] = -1;
    for(i = 0; i < n; i++){found = 0;
    for(j = 0; j < f; j++){
    if(frames[j] == pages[i]){count++;time[j] = count;
    found = 1;break;}}
    if(found == 0){
    min = time[0];
    pos = 0;
    for(j = 0; j < f; j++){
    if(frames[j] == -1){pos = j;break;}
    if(time[j] < min){
    min = time[j];
    pos = j;}}
    frames[pos] = pages[i];
    count++;
    time[pos] = count;
    fault++;}}
    printf("Total Page Faults = %d\n", fault);
    return 0;
}
```

OUTPUT :

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano lru.c
(kali@YUVARAJK)-[~]
$ gcc lru.c -o lru
(kali@YUVARAJK)-[~]
$ ./lru
Enter Number of Pages: 2
Enter Reference String:
5
Enter Number of Frames: 3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@YUVARAJK)-[~]
$
```

SHELL PRROGRAMMING : LRU

The screenshot shows a Kali Linux terminal window with the following content:

```

kali@YUVARAJIK: ~
Session Actions Edit View Help
GNU nano 9.0 lru.sh
#!/bin/bash
echo "LRU Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "LRU Algorithm Executed"

# LRU Algorithm Implementation
# 1. Initialize a list to hold the pages in memory
# 2. Iterate through the reference string
# 3. For each page, check if it is in the memory list
# 4. If not, add it to the list
# 5. If yes, update its position in the list
# 6. Print the final state of the memory list

# Example execution
# ./lru.sh
# LRU Page Replacement Demonstration
# Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
# LRU Algorithm Executed

```

The terminal window has a menu bar with "Session", "Actions", "Edit", "View", and "Help". The status bar at the bottom displays various keyboard shortcuts for editing and navigation, such as "Help", "Write Out", "Where Is", "Cut", "Execute", "Justify", "Location", "Go To Line", "Undo", "Redo", "Set Mark", "Copy", "To Bracket", "Where Was", "Previous", and "Next".

OUTPUT :

```

kali@YUVARAJK: ~
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
kali@YUVARAJK:~]
$ nano lru.c
kali@YUVARAJK:~]
$ gcc lru.c -o lru
kali@YUVARAJK:~]
$ ./lru
Enter Number of Pages: 2
Enter Reference String:
5

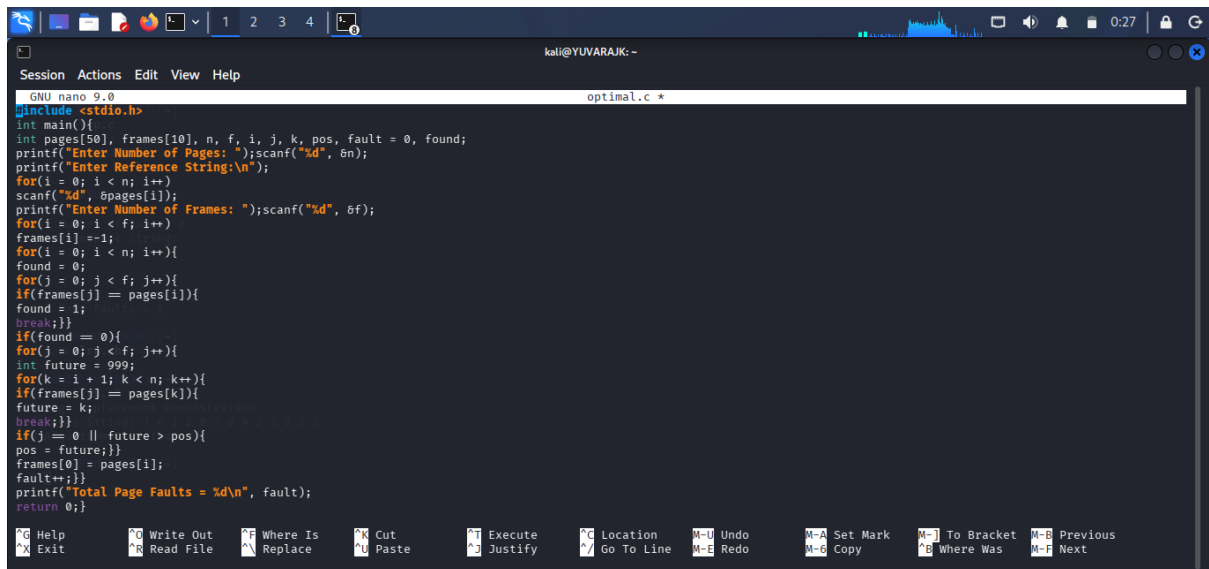
3
Enter Number of Frames: 1
Total Page Faults = 2

kali@YUVARAJK:~]
$ nano lru.sh
kali@YUVARAJK:~]
$ bash lru.sh
LRU Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
LRU Algorithm Executed

kali@YUVARAJK:~]
$

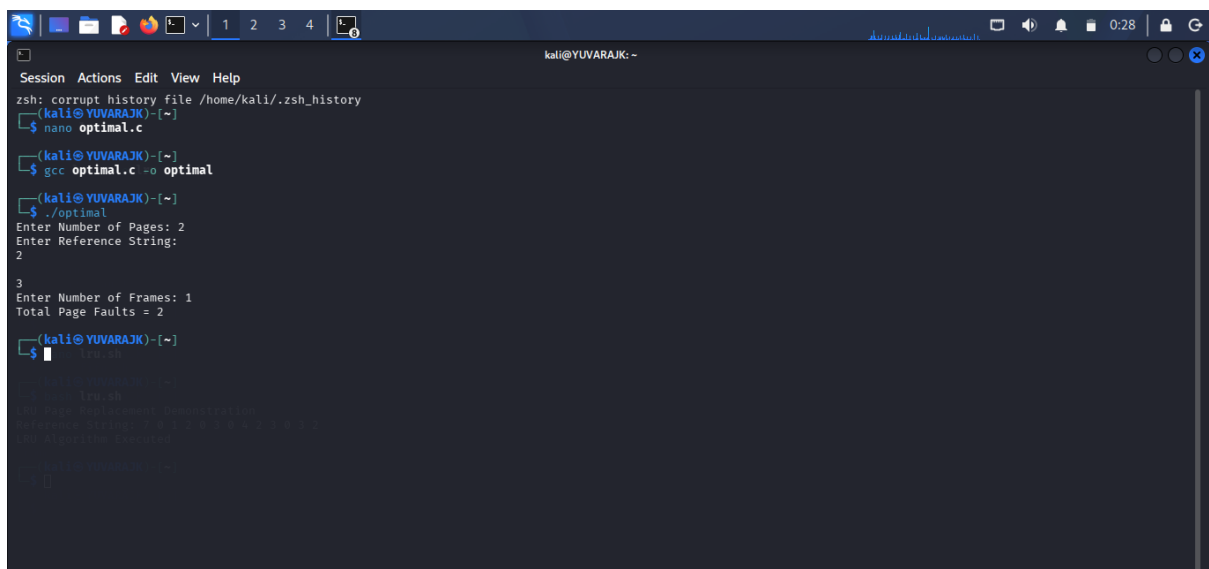
```

C PROGRAMMING :OPTIMAL PAGE



```
GNU nano 9.0 optimal.c *
#include <stdio.h>
int main(){
    int pages[50], frames[10], n, f, i, j, k, pos, fault = 0, found;
    printf("Enter Number of Pages: ");scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &pages[i]);
    }
    printf("Enter Number of Frames: ");scanf("%d", &f);
    for(i = 0; i < f; i++){
        frames[i] = -1;
    }
    for(i = 0; i < n; i++){
        found = 0;
        for(j = 0; j < f; j++){
            if(frames[j] == pages[i]){
                found = 1;
                break;
            }
        }
        if(found == 0){
            for(j = 0; j < f; j++){
                int future = 999;
                for(k = i + 1; k < n; k++){
                    if(frames[j] == pages[k]){
                        future = k;
                        break;
                    }
                }
                if(j == 0 || future < pos){
                    pos = future;
                }
                frames[j] = pages[i];
                fault++;
            }
        }
        printf("Total Page Faults = %d\n", fault);
    }
    return 0;
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK:~)
$ nano optimal.c
(kali@YUVARAJK:~)
$ gcc optimal.c -o optimal
(kali@YUVARAJK:~)
$ ./optimal
Enter Number of Pages: 2
Enter Reference String:
2
3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@YUVARAJK:~)
$
```

SHELL PROGRAMMING :



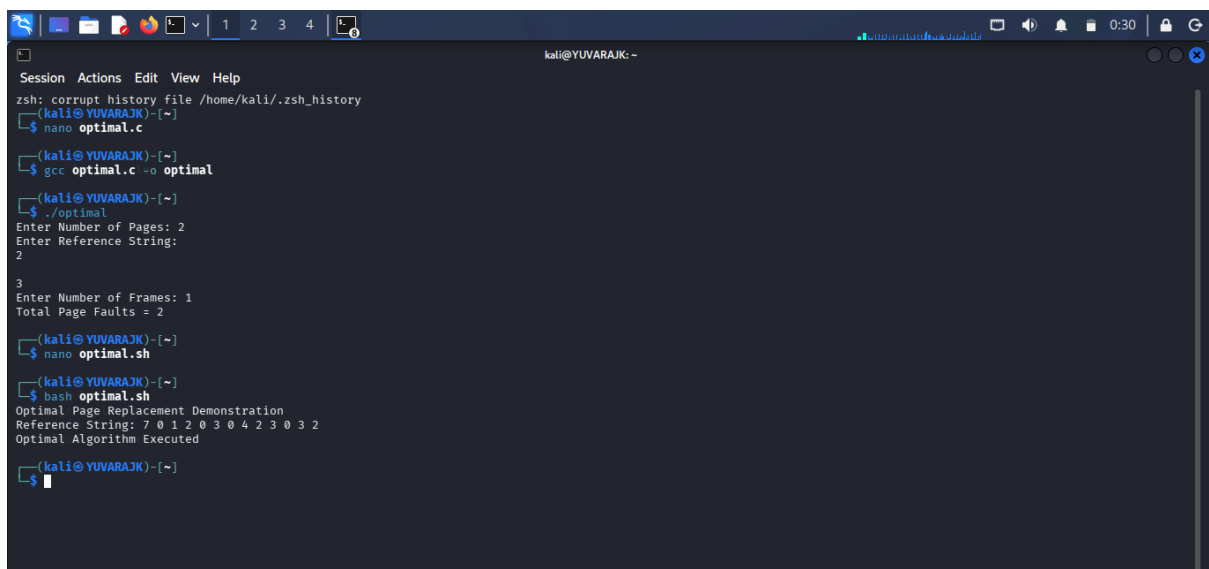
```
GNU nano 9.0 optimal.sh
#!/bin/bash
echo "Optimal Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "Optimal Algorithm Executed"

#
# Enter Number of Pages: 2
# Enter Reference String:

#
# Enter Number of Frames: 1
# Total Page Faults = 2

#
# nano optimal.c
# gcc optimal.c -o optimal
#
# nano optimal.sh
# bash optimal.sh
#
# Optimal Page Replacement Demonstration
# Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
# Optimal Algorithm Executed
#
# nano optimal.sh
#
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano optimal.c
(kali@YUVARAJK)-[~]
$ gcc optimal.c -o optimal
(kali@YUVARAJK)-[~]
$ ./optimal
Enter Number of Pages: 2
Enter Reference String:
2
3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@YUVARAJK)-[~]
$ nano optimal.sh
(kali@YUVARAJK)-[~]
$ bash optimal.sh
Optimal Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Optimal Algorithm Executed
(kali@YUVARAJK)-[~]
$
```